温州理工学院

《数学建模与实验》课程大论文

（参加 2022 年华中杯数学建模竞赛团队参赛作品）

选做题号：__A__（填写 A 或 B 或 C）

	参赛队员 1	参赛队员 2	参赛队员 3
姓名	於豪祺	王帅骏	应维
学号	20219130217	20219111103	20219111318
学院	数据科学与人工智能学院	数据科学与人工智能学院	数据科学与人工智能学院
班级	20 应用统计二班	20 计算机一班	20 计算机三班
电话	19858817237	18757816912	19941231600
Email	1768110051@qq.com	1243462510@qq.com	2062710664@qq.com

温州理工学院数据科学与人工智能学院

《数学建模与实验》课程组制

2022 年 4 月 15 日

基于遗传算法的分拣系统优化问题

摘要

分拣环节是配送中心工作流程的一个环节，分拣系统的工作效率影响了配送中心的整体性能。本文通过建立数学模型，从订单分批、货品摆放、工人指派，三个方面对分拣系统进行优化。

对于第一问，首先我们根据提供的订单信息，选出每一批次中的两订单（i 和 j），计算出两订单中具有相同货品种类的货品数量（N_k），再用得出的货品数量（N_k）除以订单 i 中的商品数量（count(i)），计算出订单适应度。依据求出的适应度建立基于遗传算法的优化模型，选择商品数量最大的订单作为初始种群，选取未分配订单中适应度最大的订单加入初始种群，判断该订单加入初始种群后是否满足加入条件，如果满足则确认加入，依次加入直到不满足加入需求。得到订单“D0148”,“D0674”,“D0726”,“D0613”,“D0497”,“D0601”,“D0337”,“D0128”为第一批次，然后继续进行下一批次的初始种群选择和其它种群的加入。通过编程遍历求解得出，将订单分 73 批的时候效率最高，具体分批方案可见文件 result1.csv。

对于第二问，针对设计货品摆放位置，使得同一订单中货品摆放尽量集中。首先研究某一批次中每个订单中货架与分拣总距离的关系，建立拣选距离模型。接着通过模拟退火算法进行求解，设置初始温度、温度下限、温度下降率和颗粒总数，并对货架距离进行迭代，产生新解，例：第一批次中“P0897”放在“1 号货架”,“P0078”放在“200 号货架”。通过编程遍历求解得出最优货品摆放方案，具体方案可见文件 result2.csv。

对于第三问，对于得出的订单货品拣选距离要分配工人进行拣选，需要工人尽快完成任务，并且每个分拣工的运动距离尽可能均衡。首先根据工人与订单之间的关系，建立指派模型。通过研究分拣工完成一个订单后切换至下一订单的距离与分拣工完成所有订单的总距离的关系，建立出距离优先模型，依次选取剩余订单中距离最长的，指派给分拣工，如果指派的分拣工数量超出了给定的数量，则返回给第一个分拣工，例：第一批次的 1 号工人需要完成“D0148”,“D0128”两个订单。通过编程遍历求解得出最优分拣工指派方案，具体方案可见文件 result3.csv。

关键词：适应度；初始种群；遗传算法；模拟退火算法；距离优先

目录

摘要 2
一 问题重述 4
1.1 问题背景 4
1.2 需要解决的问题 4
二 问题分析 4
2.1 问题一的分析 4
2.2 问题二的分析 5
2.3 问题三的分析 5
三 模型假设 5
四 符号说明 6
五 模型的建立与求解 6
5.1 问题一的模型建立与求解 6
5.1.1 适应度的定义与计算 6
5.1.2 遗传算法的模型建立 7
5.2 问题二的模型建立与求解 10
5.2.1 拣选距离模型 10
5.2.2 模拟退火算法求出货架编号 11
5.1 问题三的模型建立与求解 15
5.1.1 指派模型 15
5.1.2 距离优先模型 15
六 模型分析 18
七 模型的评价、改进和推广 18
7.1 问题一模型的评价、改进和推广 18
7.2 问题二模型的评价、改进和推广 18
7.3 问题三模型的评价、改进和推广 18
八 参考文献 19
附录 19

一 问题重述

1.1 问题背景

分拣环节是配送中心工作流程的一个环节，其关键性地影响着配送中心整体工作性能，根据统计，分拣环节的工作量在配送中心工作量中占比 60%，该环节成本占整个配送成本的 50%-75%。分拣环节中的订单分批，货品摆放位置，指派分拣任务都影响着分拣工作的效率。

1.2 需要解决的问题

某电商公司将要面临货架数量不够的问题，需要尝试提高分拣效率。不需要考虑货架上可以放置货品的件数。

问题 1：当日有 923 个订单，923 个订单中货品种类有 1941 种。要对这些订单进行分批处理，已知给定的货架数为 200，要求每个批次所包含的货品种类不超过给定的货架数（一个货架只能放置一种货品）。需要给出相应的分批方案使批次数达到最小，从而有更高的工作效率。

问题 2：在问题 1 得出最优分批方案的基础上，确定每批次中，每种货品的摆放位置。已知 200 个货架呈直线排列，且货架的序号从左到右为 1 到 200。将货品 i 所在的货架序号记为 $S(i)$，将拣选距离定义为

$$d(O_k) = \max_{i \in O_k} S(i) - \min_{i \in O_k} S(i)$$

其中 O_k 为某一批次中的第 k 个订单。需要给出相应的货品摆放方案使各批次分拣任务的拣选距离达到最小。

问题 3：在问题二得出每个批次的最优货品摆放方案的基础上，确定每个分拣工人的分拣任务。已知下列条件 1.分拣工人数为 5；2.分拣工工作时的运动路线不会冲突;3.一个分拣工在同一时间不能同时处理多个订单；4.分拣工在工作时始终是匀速运动且速度都相同，只考虑除匀速运动至相应货架的时间，其余全部忽略；5.分拣工只要经过该订单中某一货品所在的货架一次，就可以视为已经拣选到该货品；6.分拣工将同一订单的货品放入同一拣货框中，不需要考虑其容积，将一个订单的货品拣选完之后将拣选框换新，忽略这其中所花的时间；7.分拣任务开始时，每位分拣工需要按照任务单的订单顺序，完成分拣工作；8.每一批分拣任务开始时，所有分拣工从第一个货架开始当前的分拣工作，每位分拣工完成一个订单后可以在结束的位置立即开始下一个分拣工作，在完成这一批次所有订单后分拣工需要回到 1 号货架；9.不考虑货架大小，两个相邻的货架之间的距离为单位 1，且分拣货物的时间与运动距离呈正比。结合以上条件，给出每个批次的指派结果和分拣工的订单处理顺序，从而使任务完成的时间最短以及每个分拣工的运动距离尽量均衡。

二 问题分析

2.1 问题一的分析

设计一个分批算法，使得效率最高，也就意味着他的批次最少。当前已治货架数量为固定，且商品种类不得多于货架数量，那么在要求批次最少的

前提下，我们在同一批次下，应尽可能的将一商品的所有订单放在一批次。

但是，题目要求同一订单的商品不可拆分，故我们不能脱离订单来单独考虑商品。

因此，我们可以想到运用遗传算法，引入一个全新的标量量——“适应度”，通过比较当前订单与剩余订单的“适应度”来判断是否加入，取与当前批次中适应度最高的订单加入该批次，尽可能的在固定的货物种类下，使更多的订单加入到当前批次。

综上，我们可将解题过程分为以下步骤：(1)将所给的附件读入到 python 中，并且以字典的形式储存，并按订单数量降序排列。(2) 依次遍历数据，通过比较适应度来判断是否加入，若该批次商品数量超过货架数量，则跳到下一批次。

2.2 问题二的分析

在问题一已经分好批次的基础上，我们需要将每批次里的商品进行排序，使得在每批次里拣选距离最短，以及总的拣选距离也是最短的。

由于是考虑每一批次，因此我们就要遍历每一批次，并且在每一批次中进行操纵。从每批次里 200 左右的商品对其进行排序，我们可以考虑到模拟退火算法。

模拟退火算法可以从全局出发，通过概率，找到最优解。该题的主要步骤为：(1) 通过遍历第一问中的所有批次，把每个订单的商品号拿出来。(2) 生成随机位置编号，然后找出该批次中订单与其商品对应的位置，计算它的拣选距离。

2.3 问题三的分析

在问题二的基础上，我们已经知道了每一批次中取每一订单的最优距离，我们需要在这基础上进行排序，当员工完成当前订单，此时大约处在货架靠后位置，再从货架后部往前取货，以保证充分利用距离。

因此我们可以用长度优先算法，通过对订单长度的降序排序，从高到低依次分配给 5 位员工，然后取接下来一订单分配给第一位员工，依次类推，保证时间总和最短。

综上，我们可以将解题过程分为以下步骤：(1)将第二题得到的完成每个订单的长度导入。(2)将长度降序排序，依次分配给员工。

三 模型假设

1. 货架只考虑货架的个数，每一个货架只能存放一种货品，不考虑货架的容积和载重限制。
2. 因为取货时间不考虑，认为距离长短代表拣选效率。消耗时间等价于移动距离总和。
3. 只考虑分拣环节中的订单分批，货品摆放位置，指派分拣任务对分拣工作的影响。
4. 本模型不考虑货物的种类，以及不同商品的重量、包装等对摆放以及般运的影响。
5. 分拣工在分拣过程中互相不会干扰，即运动路线不会冲突，也可以同时

拣选同一货架上的货品。

四 符号说明

符号	说明	单位
F_i	订单相似度表量化表示	/
K	批次	次
D	最大分拣距离	米
d_i	任意两商品分拣距离	米
S_i	货架序号	/
S(n)	货品编号	/
W	工人编号	/
T_i	判断订单 i 是否被指派	/
M_W	编号 w 的工人移动的距离	/
Count ()	计算元素的个数	/

五 模型的建立与求解

5.1 问题一的模型建立与求解

5.1.1 适应度的定义与计算

拣选位分配优化问题是一个典型的旅行商问题，是 NP 难问题，只能使用启发式算法求解。因为目标模型存在非常多组的可行解，同时也会有很多局部最优解，所以求解算法要求有良好的全局搜索能力和局部搜索能力，使用改进的遗传算法就可以解决这一问题[1]。

遗传算法具有良好的全局搜索能力，可以快速地将解空间中的全体解搜索出，而不会陷入局部最优解的快速下降陷阱。并且利用它的内在并行性，可以方便地进行分布式计算，加快求解速度。

通过遗传算法，我们在所有订单中搜索，与订单适应度进行比

较，符合遗传规律“适者生存，不适者淘汰”，将满足条件的订单拿出来加入该批次。

订单适应度(F_i)是指订单包含的各属性相似程度的数值度量，在本题目中，订单中包含若干个商品，商品的编号即为该订单的属性。在同一批次（k）中，我们要找出两个订单（i 和 j）相同标号数值，即在两订单中，具有相同商品编号的数量（N_k）。适应度的计算公司：

$$F_i = \frac{N_k}{count(i)} \tag{1}$$

其中 count(i)表示在订单 i 中的商品数量。

以下是在不同一批次中的商品适应度。

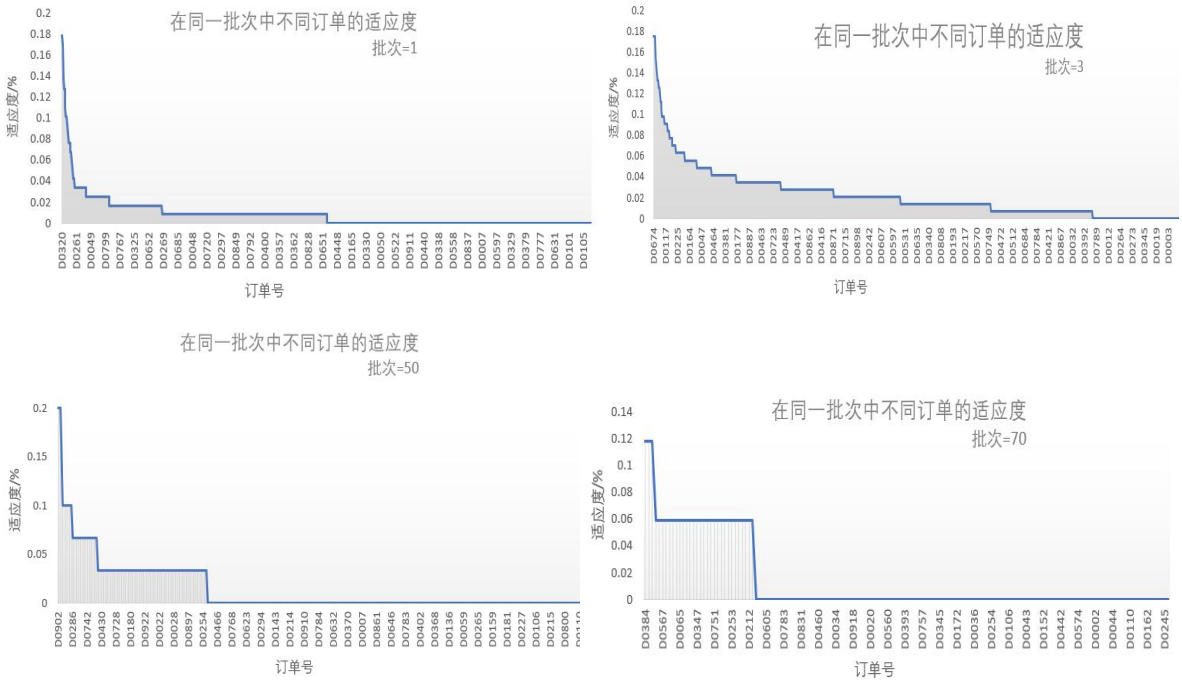

图 1 不同批次的订单适应度

5.1.2 遗传算法的模型建立

遗传算法是根据遗传学机理的物种进化过程和 生物进化论的自然选择模拟的一种计算模型，通过 模拟自然进化的过程来对最优解进行搜索。遗传算法属于一种随机优化的算法，但不是普通的随机 结果验证，而是通过对于染色体的评价和染色体基 因的作用，使用已知的所有信息来指导搜索，寻求最优解[2]。

由于模型中各个订单的数量不是唯一的[2]，我们需要先对初始的数据进行处理，从订单中商品数量最多的订单作为初始种群，然后不断扩大，直至不符合条件。在下一批次中，我们任然将剩余订单中，商品数量最多的作为初始总群，以此可以使得每批次中的商品数量尽可能的多，并且使商品适应度的值更加可靠。

以下是该模型的步骤：

a) 统计各个订单中商品的数量，并且按照数值大小降序排列存储在计算机中。

b) 选取当前未安排订单中商品数量最大的订单加入到该批次中，以此做为初始种群。

c) 计算当前订单与所有未分配订单的商品适应度，取其中适应度最大的订单。

d) 判断将该订单加入“种群“后是否还满足要求，若满足则加入初始种群并且继续依次加入商品适应度第二大的订单，直到不满足要求。

e) 当当前批次已经饱和时，进入下一批次，且依旧以未分配订单中商品数量最大的作为初始种群。

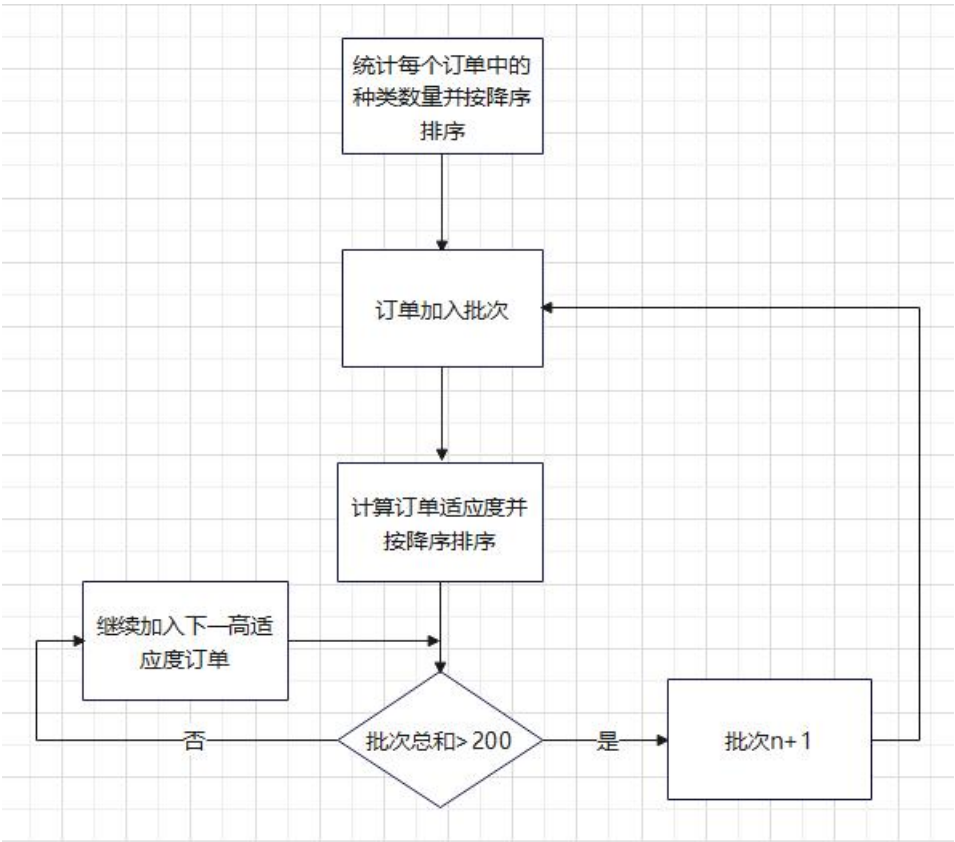

图 2 遗传算法求解流程

将上述所有模块汇总建立遗传算法模型（如图 2 遗传算法求解流程所示），并用 python 语言编写结构化程序进行求解，并将输出结果保存在 result1.csv 文件中（代码见附录一和支撑文件下的 问题一.py 文件），最终部分结果见下图（第一列为 OrderNo,第二列为 GroupNo）。

OrderNo	GroupNo	D0680	3	D0133	9	D0561	73
D0148	1	D0459	3	D0767	9	D0574	73
D0674	1	D0029	3	D0481	9	D0595	73
D0726	1	D0175	4	D0171	9	D0628	73
D0613	1	D0565	4	D0741	9	D0800	73
D0497	1	D0432	4	D0678	9	D0840	73
D0601	1	D0880	4	D0258	9	D0002	73
D0337	1	D0115	4	D0194	10	D0003	73
D0128	1	D0690	4	D0851	10	D0005	73
D0190	2	D0149	4	D0894	10	D0009	73
D0320	2	D0145	5	D0089	10	D0026	73
D0670	2	D0164	5	D0084	10	D0044	73
D0882	2	D0225	5	D0102	10	D0045	73
D0474	2	D0204	6	D0060	11	D0091	73
D0505	2	D0246	6	D0049	11	D0092	73
D0413	2	D0464	6	D0208	11	D0105	73
D0591	2	D0124	6	D0203	11	D0110	73
D0498	2	D0525	6	D0063	12	D0112	73
D0486	2	D0283	7	D0625	12	D0126	73
D0572	2	D0458	7	D0187	12	D0146	73
D0540	2	D0744	7	D0731	12	D0158	73
D0825	2	D0719	7	D0737	12	D0162	73
D0468	2	D0189	7	D0618	12	D0169	73
D0612	2	D0006	8	D0039	12	D0188	73
D0401	2	D0621	8	D0755	12	D0205	73
D0409	2	D0553	8	D0599	12	D0212	73
D0433	2	D0252	8	D0131	13	D0221	73
D0771	2	D0504	8	D0909	13	D0243	73
D0031	3	D0416	8	D0718	13	D0245	73
D0541	3	D0257	8	D0339	13	D0260	73

图 3 问题一部分结果

5.2 问题二的模型建立与求解

5.2.1 拣选距离模型

（1）确定公式

在本题中，根据题意，可将已经分好批次的订单化作一个个独立的模块，以一个批次为一个模块，各模块相互间互不干涉，故只要求在单一批次中拣选距离最短的即可。

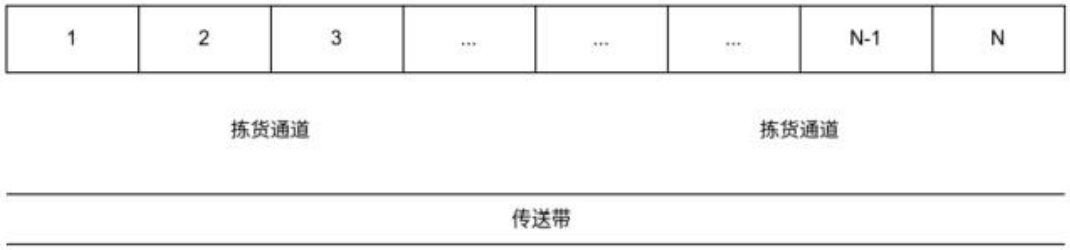

图 4 货架示意图

分拣工拣选货品时，同一订单所含货品的位置越集中，移动距离越短，拣选效率越高。假设所有货架等距排列在一条直线上，按照摆放位置的顺序，货架的序号依次为 1，2，3，4，5，6……N，货品所在货架的序号为 S(n)。在一订单中某一商品的货架号为 O_k。

因此，在单一批次中，任意两商品的分拣距离 di （i 表示商品号)的公式为：

$$d_i = O_{K-1} - O_K \tag{2}$$

我们定义，在订单的分拣距离为同一订单中货架号最大与最小的差值。故在批次中，某一订单的分拣距离 Di 的计算公式应该为：

$$D_i = \max(O_K) - \min(O_K) \tag{3}$$

(2)采用以上公式，我们可以清楚的看出，要想使得 D 取得最小，我们就要使 max(Ok)尽可能的小，这样拣选距离总和才会最小。

$$\sum_{i=0}^{k} \min(D_i) \tag{4}$$

（3）采用 matlab 编写，将该部分代码放入模拟退火算法中，具体的代码见附录二。

5.2.2 模拟退火算法求出货架编号

模拟退火算法（simulated annealing，SA）来源于固体退火原理，是一种基于概率的算法。算法思想为：先从一个较高的初始温度出发，逐渐降低温度，直到温度降低到满足热平衡条件为止。在每个温度下，进行 n 轮搜索，每轮搜索时对旧解添加随机扰动生成新解，并按一定规则接受新解[4]。

模拟退火算法本质是双层循环，外层循环控制温度由高向低变化，温度计算公式 ， 为取值在[0, 1]上的温度衰减系数，如 0.95；内层循环中，温度固定，对旧解添加随机扰动得到新解，并按一定规则接受新解。通过这种

算法，不管函数形式多复杂，模拟退火算法更有可能找到全局最优解。

以下为模拟退火算法在本题的运行流程图：

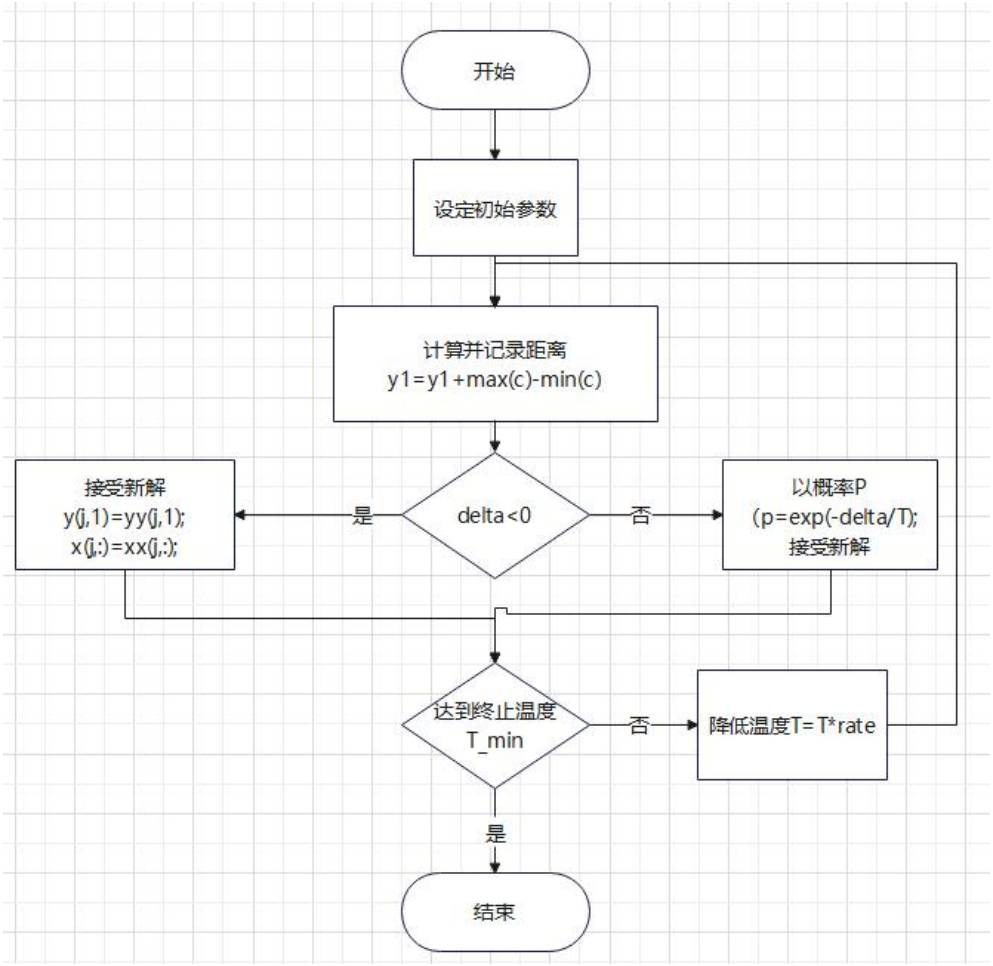

图 5 模拟退火算法流程图

在本题中，我们通过设置初始温度，温度下限，温度下降率和颗粒总数等，通过不断的迭代，产生新解，去判断新解是否符合，如果符合则加入，不符合继续进行下一次循环。

将上述所有模块汇总，并用 matlab 语言编写结构化程序进行求解，并将输出结果保存在 result2.csv 文件中（代码见附录二和支撑文件下的 quextion2.m 文件），最终部分结果见下图（第一列为 ItemNo,第二列为 GroupNo，第三列为 ShelfNo）。

ItemNo	GroupNo	ShelfNo	P0879	1	136	P0206	34	84	P1252	73	59
P0044	1	93	P0880	1	72	P0207	34	87	P1270	73	129
P0053	1	113	P0881	1	30	P0211	34	46	P1276	73	33
P0063	1	106	P0888	1	194	P0216	34	22	P1308	73	158
P0065	1	196	P0906	1	3	P0226	34	200	P1363	73	74
P0076	1	152	P0909	1	186	P0239	34	45	P1394	73	47
P0077	1	86	P0910	1	41	P0247	34	194	P1406	73	50
P0078	1	134	P0911	1	167	P0249	34	164	P1417	73	180
P0097	1	11	P0920	1	137	P0250	34	161	P1425	73	112
P0106	1	193	P0922	1	142	P0251	34	192	P1482	73	108
P0141	1	29	P0936	1	172	P0252	34	122	P1520	73	76
P0165	1	183	P0938	1	100	P0255	34	18	P1535	73	154
P0167	1	78	P0941	1	130	P0257	34	111	P1540	73	3
P0171	1	36	P0943	1	198	P0340	34	183	P1570	73	72
P0177	1	184	P0944	1	19	P0341	34	72	P1586	73	192
P0191	1	55	P0945	1	192	P0343	34	158	P1589	73	24
P0194	1	32	P0947	1	141	P0355	34	43	P1609	73	160
P0209	1	45	P0948	1	103	P0363	34	76	P1617	73	75
P0211	1	150	P0949	1	12	P0379	34	166	P1630	73	124
P0251	1	26	P0950	1	69	P0381	34	15	P1675	73	88
P0252	1	169	P0981	1	51	P0426	34	174	P1689	73	136
P0277	1	170	P1006	1	139	P0427	34	129	P1728	73	172
P0279	1	164	P1011	1	154	P0446	34	139	P1755	73	171
P0297	1	124	P1017	1	133	P0449	34	130	P1788	73	148
P0302	1	75	P1018	1	158	P0491	34	28	P1800	73	106
P0305	1	76	P1019	1	145	P0504	34	186	P1802	73	4
P0307	1	53	P1020	1	120	P0505	34	155	P1808	73	161
P0311	1	96	P1044	1	104	P0523	34	179	P1931	73	61
P0312	1	107	P1045	1	79	P0525	34	35	P1940	73	127
P0364	1	168	P1046	1	42	P0538	34	156	P1941	73	123

图 6 问题二的部分结果

获取每个商品在每一批次的货架号后，我们可以计算出每个订单的分拣距离和所有订单的分拣距离和。通过 Python 语言（代码见附录三）的求解，部分结果可见下图，具体结果可在支撑文件下拣选距离.csv 查看。

OrderNo	d
D0148	197
D0674	195
D0726	196
D0613	197
D0497	167
D0601	164
D0337	194
D0128	192
D0190	198
D0320	193
D0670	189
D0882	198
D0474	164
D0505	195
D0413	171
D0591	193
D0498	170
D0486	180
D0572	188
D0540	176
D0825	168
D0468	199
D0612	156
D0401	180
D0409	173
D0433	181
D0771	161
D0031	198
D0541	180
D0894	179
D0089	188
D0084	195
D0102	196
D0060	197
D0049	197
D0208	196
D0203	186
D0063	199
D0625	185
D0187	184
D0731	196
D0737	196
D0618	194
D0039	193
D0755	180
D0599	143
D0131	194
D0909	196
D0718	184
D0339	192
D0363	199
D0417	193
D0518	187
D0325	182
D0754	185
D0517	189
D0528	166
D0056	194
D0280	199
D0406	192
D0620	169
D0810	182
D0323	186
D0461	185
D0261	197
D0781	189
D0269	199
D0761	195
D0462	169
D0774	196
D0901	171
D0456	135
D0319	180
D0577	184
D0453	163
D0691	171
D0545	174
D0014	196
D0224	187
D0192	187
D0141	189
D0785	186
D0231	194
D0281	199
D0183	180
D0696	192
D0176	182
D0598	169
D0697	184

图 7 各订单的分拣距离

通过画出订单号与距离之间的图像，我们可以看出，在同一批次中，不同订单的分拣距离大致相同。以下是部分订单与距离的图像：

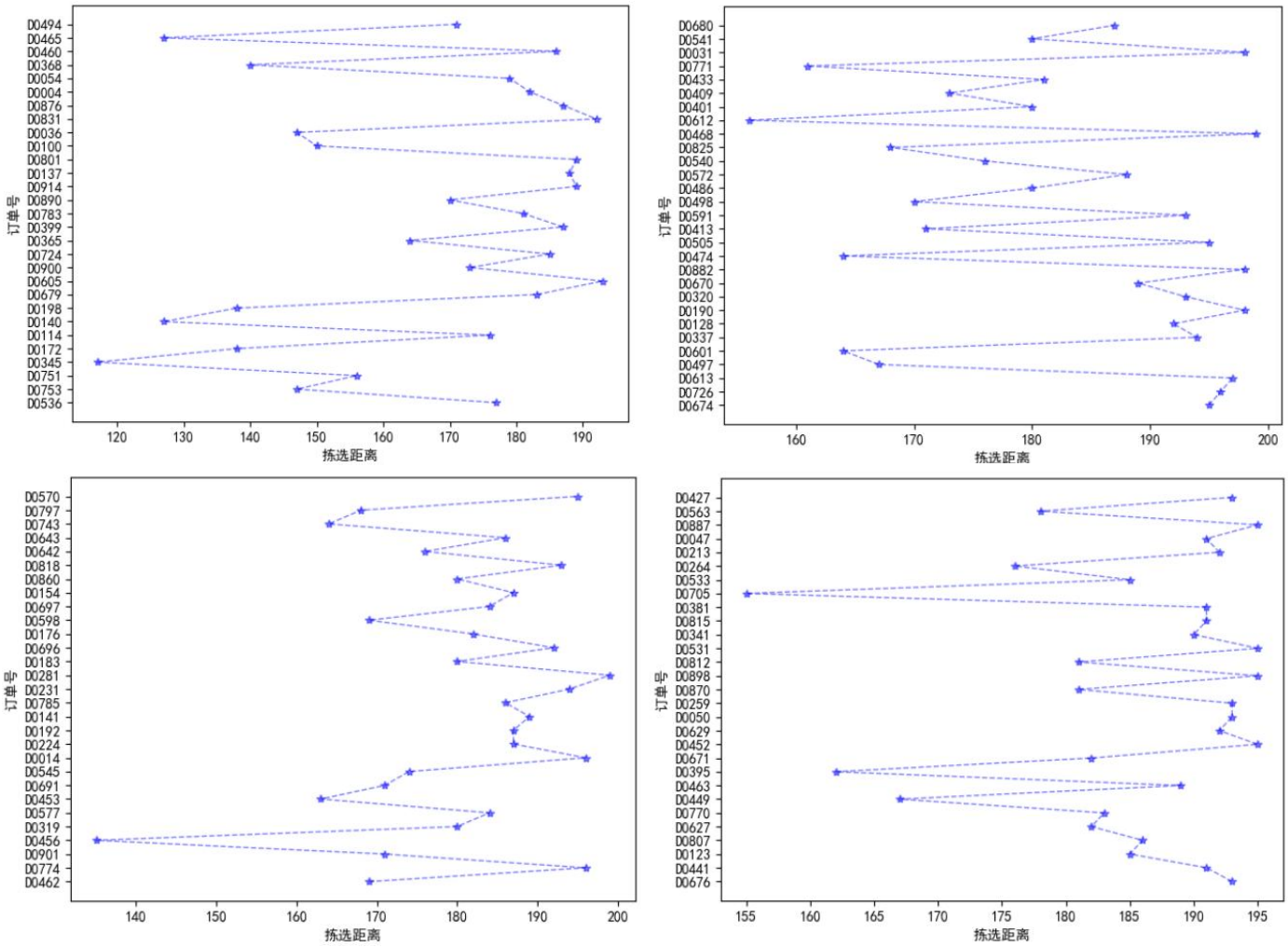

图 8 拣选距离与订单号的折线图

通过以上算法，我们求得拣选距离总和 D 为 155903。

5.1 问题三的模型建立与求解

5.1.1 指派模型

通过第二问的求解，我们已经求出里每个订单的拣选距离（di），我们可以通过这个数据来对本题进行求解。

每个工人都从一号出发，并且在同一批次中，相邻订单间工人不需要返回起点位置，使得能够尽快完成任务，且运动距离尽可能均衡，故我们要使得工人走的距离总和最短，就要让每一个工人在每一次任务中走的距离应该尽可能地相似。假设在一次任务中，引入一个新的变量 T，使得每个订单（i）拥有这个变量，即 Ti，该变量满足：

$$T_i = \begin{cases} 1 & \text{该商品已经被指派} \\ 0 & \text{该商品未被指派} \end{cases} \tag{5}$$

5.1.2 距离优先模型

对与上述指派模型，继续对工人拣选进行研究。假设工人 w 的移动距离 Mw，则 Mw 应该满足：

$$M_w = \max(d_i) \tag{6}$$

$$s.t. \quad T_i = 0 \tag{7}$$

通过该算法，依次选取剩余 d_i 中最大的，将其指派给工人 W，然后我们就将这一订单 T_i 改为 1，W 也增加一个。若 W 超过了现有工人，则 W 回到第一个工人。

运用上述算法，通过 Python 语言（代码见附录四）的求解，部分结果可见下图，具体结果在支撑文件下 result3.csv 查看。（下图第一列为 OrderNo，第二列为 GroupNo，第三列为 WorkNo，第四列为 TaskNo）

OrderNo	GroupNo	WorkNo	TaskNo
D0148	1	1	1
D0613	1	2	1
D0726	1	3	1
D0674	1	4	1
D0337	1	5	1
D0128	1	1	2
D0497	1	2	2
D0601	1	3	2
D0468	2	1	1
D0190	2	2	1
D0882	2	3	1
D0505	2	4	1
D0320	2	5	1
D0591	2	1	2
D0670	2	2	2
D0572	2	3	2
D0433	2	4	2
D0486	2	5	2
D0401	2	1	3
D0540	2	2	3
D0409	2	3	3
D0413	2	4	3
D0498	2	5	3
D0825	2	1	4
D0474	2	2	4
D0771	2	3	4
D0612	2	4	4
D0029	3	1	1
D0031	3	2	1
D0152	73	4	6
D0122	73	5	6
D0168	73	1	7
D0312	73	2	7
D0279	73	3	7
D0070	73	4	7
D0002	73	5	7
D0003	73	1	8
D0005	73	2	8
D0009	73	3	8
D0026	73	4	8
D0044	73	5	8
D0045	73	1	9
D0091	73	2	9
D0092	73	3	9
D0105	73	4	9
D0110	73	5	9
D0112	73	1	10
D0126	73	2	10
D0146	73	3	10
D0158	73	4	10
D0162	73	5	10
D0169	73	1	11
D0188	73	2	11
D0205	73	3	11
D0212	73	4	11
D0221	73	5	11
D0243	73	1	12
D0245	73	2	12
D0260	73	3	12

图 9 问题三部分结果

我们通过 Python 画图软件，模拟每个每个工人移动的折线图，可以得到以下图形。

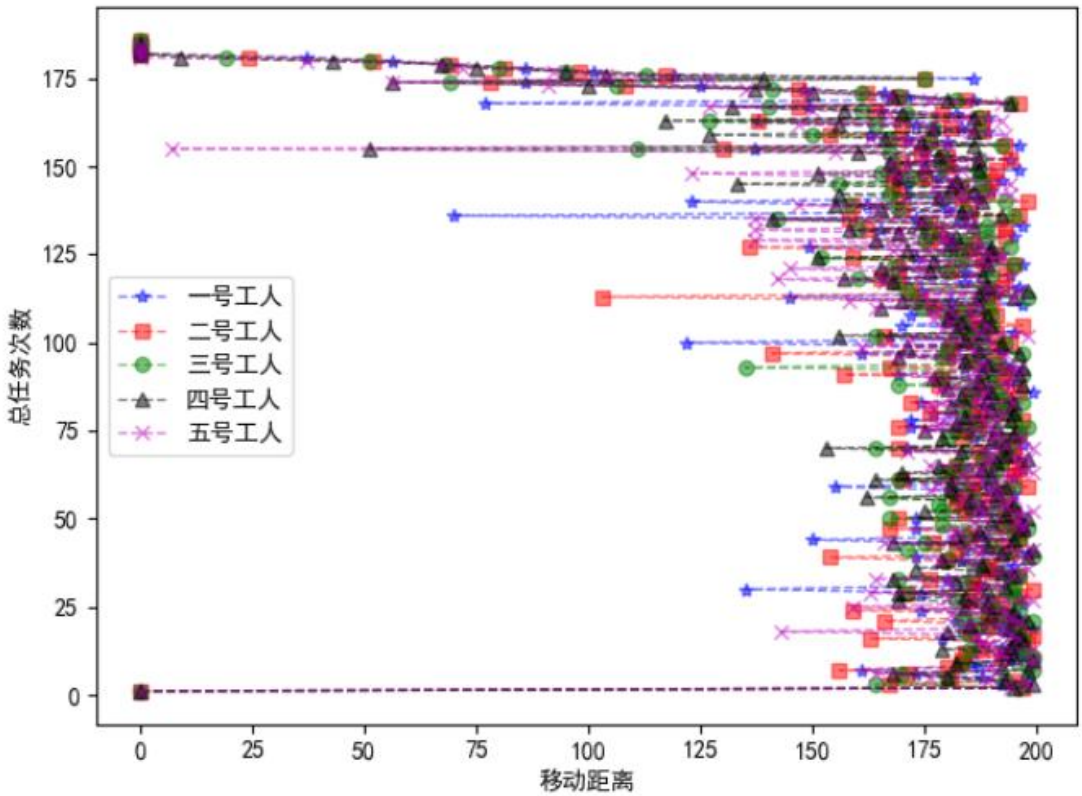

图 10 工人移动距离

.通过观察上图发现，五个工人在每一次任务中移动的距离较为均衡且集中，并且依靠长度优先的原则，在前面的任务会先运动到货架编号较靠后的货架，工人在执行下一任务时也会较快的到达起始位置，依次来保证工人能够较快的完成任务。

再将单个工人的运动距离画出（见图 11 单一工人移动距离），我们发现工人只需要在前面和最后的任务中去走大量的货架，在中间的任务中，工人的移动距离摆动，并且逐渐减少，直到最后距离变成 0。

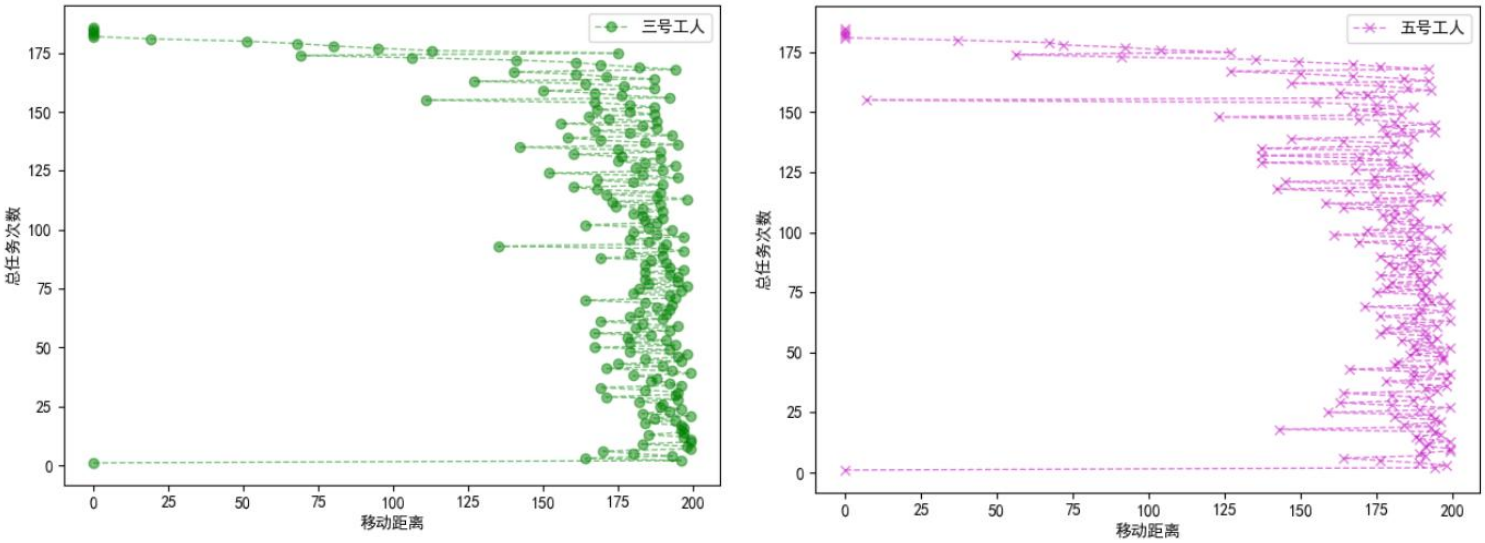

图 11 单一工人移动距离

六 模型分析

1. 局部最优解造成的货品摆放位置的误差，模拟退火算法本身会导致其收敛成局部最优解。为了避免模型陷入局部最优解，在程序运行的工程中需要人为进行干预，检查结果是否为局部最优解或缺解

2. 对于工人指派的实际情况的误差，虽然题目中表示不需要考虑除分拣工完成任务之外所花的时间，但是这些时间必然存在。与实际情况不符

七 模型的评价、改进和推广

7.1 问题一模型的评价、改进和推广

该模型通过计算订单之间的适应度，在降序情况下订单的适应度越高，代表重合率越高，所需要用到的货架数量 N 越少，则可一次取的订单数越多。因此，我们将采取遗传算法，建立适应度模型，更好量化订单的货品种类，因此可以通过计算适应度来一次取尽量多的订单，从而很好地减少批次，提高效率。问题一模型采用概率化的寻优方法，不需要确定的规则就能自动获取和指导优化的搜索空间，自适应地调整搜索方向。面对没有规则大量的订单，遗传算法正好能够轻松处理，使得搜索得到的优化结果逼近全局最优解。

7.2 问题二模型的评价、改进和推广

该模型同在第一问结果基础上，考虑货架编号，遍历每个批次通过模拟退火算法分别寻找最优位置分布，每个颗粒对应一组随机生成位置编号，然后遍历批次中的订单，通过 ismember 函数匹配并抽取出其货品对应的位置，按题目给出的距离公式计算，目标函数为总距离最小化对位置分布进行寻优，每个批次的位置寻优是单独的，然后判断是否更新最优，不是最优则更新。最后结果保存在 P 矩阵中，与每个批次的货品种类 U 矩阵对应，最终结果见 PP 矩阵。模拟退火算法中会以一定的概率跳出这样的局部最优解，以寻找目标函数的全局最优解。诺在遍历完全之前已经达到了一种最优解，模拟退火算法会以一定的概率跳出这个解，这样在一定程度上增加了寻找到全局最优解的可能性。当该模型结合其他算法，有更加广泛的应用。

7.3 问题三模型的评价、改进和推广

该模型在前两问的基础上，对订单的批次，以及商品货架号已经知道了，通过计算可以得出各个订单的最佳拣选距离，我们将所有订单分批次，按照不同批次中的订单的拣选距离排序，给工人们优先分配长距离的任务，再分配短距离任务，使得每个工人在上一任务中的结束点可以直接去开始点，中间浪费的距离较少。然而，该模型并没有考虑到具体的货架号对于路径对工人搬运的影响，如果在一订单中，拣选距离较短，但是货架距离上一任务的终点较远，那么在该条件下，工人的用时并不是最佳的，因此，该模型优化的方向应该朝着将具体的货架号引入，通过与其余订单比较，确定工人的分配。

八 参考文献

[1]. 肖可，谢勇， 王红卫，《基于复合相似度的订单分批启发式种子算法》，华中科技大学人工智能与自动化学院，系统工程学报，2020.35（06）

[2]. 陈志新，董瑞雪，《基于改进遗传算法的自动拣选系统拣选位分配建模与优化》，北京物资学院，工业工程，2019 年 12 月第 22 卷第 6 期

[3]. 秦馨，赵剑道，《基于聚类算法的订单分批策略研究》，北京机械工业自动化研究所，制造业自动化

[4]. 谢维，周游，朱文斌，《基于改进模拟退火的登机口分配问题》，华南理工大学工商管理学院，计算机系统应用，2021.30（05）

附录

附件一

介绍：通过遗传算法求得问题一结果的 Python 代码。

```
import openpyxl
import csv


class tstExcel:
    def __init__(self, filepath):
        self.filepath = filepath
        self.wb = openpyxl.load_workbook(self.filepath)

    def get_Excel(self):
        sheet = self.wb.worksheets[0]
        test_data = []
        for i in range(2, sheet.max_row + 1):
            sub_data = {}
            for j in range(1, sheet.max_column + 1):
                sub_data[sheet.cell(1, j).value] = sheet.cell(i, j).value
            test_data.append(sub_data)
        return test_data

    def get_Or(self):
        sheet = self.wb.worksheets[0]
        test_data = {}
        sub_data = []
        for i in range(2, sheet.max_row + 1):
            if sheet.cell(i, 1).value == sheet.cell(i + 1, 1).value:
                sub_data.append(sheet.cell(i, 2).value)
            else:
                sub_data.append(sheet.cell(i, 2).value)
                test_data[sheet.cell(i, 1).value] = sub_data
                sub_data = []
```

```
        return test_data


def fitness(f1, f2, g):   # 计算适应度的值，传入两集合，集合中为 ItemNo
    n = sum(i in f1 for i in f2)
    p = n / len(f1)
    fls.update({g: p})


def add(tls, itemno):
    tls = list(tls + itemno)
    r1 = set(tls)
    if len(r1) > 200:
        flag = 0
    else:
        flag = 1
    return flag, tls


def save_excel(res):
    rows = list(zip(res.keys(), res.values()))
    print(rows)
    row_1 = ['OrderNo', 'GroupNo']
    with open('result1.csv', 'w', newline='')as csv_file:
        writer = csv.writer(csv_file)
        writer.writerow(row_1)
        writer.writerows(rows)


ex = tstExcel(r'D:\dek\附件 1：订单信息.xlsx').get_Or()
ex = dict(sorted(ex.items(), key=lambda x: len(x[1]), reverse=True))   # 按照
Item 的数量降序排列
res = {}   # 用于存储最后的结果
g = 1   # 结果的批次
ex_l = []   # 存放动态的 OrderNo
for i in ex.items():
    ex_l.append(i[0])

for i in ex.items():
    if i[0] not in ex_l:
        continue
    res.update({i[0]: g})
    fls = {}   # 用于存储当前情况下的适应度和 OrderNo
    # count = len(i[1])
```

```
    ex_l.remove(i[0])
    for j in ex_l:
        fitness(i[1], ex[j], j)
    fls = dict(sorted(fls.items(), key=lambda x: x[1], reverse=True))
    tls = i[1]
    for k in fls.items():
        flag = 0
        pd, tls = add(tls, ex[k[0]])
        if pd:
            res.update({k[0]: g})
            ex_l.remove(k[0])
        else:
            flag = 1
        if flag:
            break
    g += 1
print(res)
save_excel(res)
```

附录二：

介绍：通过模拟退火算法求出每一批次的商品摆放位置的 Matlab 代码。

```
clear
[~,~,X]=xlsread('附件1：订单信息.xlsx');
X=string(X);
X(1,:)=[];%读取附件一的内容

[~,~,Y]=xlsread('result1.csv');
Y=string(Y);
Y(1,:)=[];%读取第一题的结论
N=200       %货架数为200
S={}        %存储每批次中的订单号
ss=[]
a=Y(1,2)
z=1
for i=1:length(Y)
    if a~=Y(i,2)
        ss=unique(ss)
        S{z,1}=ss.'
        z=z+1
        ss=[]
        a=Y(i,2)
```

```
    end
        for j=1:length(X)
            if X(j,1)==Y(i,1)
                ss=[ss X(j,2)]
            end
        end
end
ss=unique(ss)
S{z,1}=ss.'

rate=0.99; %温度下降率
sum=10; %颗粒总数
T=100; %初始温度
T_min=1; %温度下界
R=[];
for i=1:length(S)
    a=find(Y(:,2)==string(i));%依次遍历每个批次
    XX=[];%记录订单及货品种类，后续调用
    for j=1:length(a)
        XX{j,1}=Y(j,1);
        XX{j,2}=X(find(X(:,1)==Y(a(j),1)),2);
    end
    %初始颗粒分布
    x=[];
    y=[];
    for j=1:sum
        c=[];
        y1=0;
        x{j,1}=randperm(N);
        x{j,1}=x{j,1}(1:length(S{i,1}));
        for k=1:size(XX,1)
            b=find(ismember(S{i,1},XX{k,2})==1); %
            c=[c x{j,1}(b)];
            if b~=0
                y1=y1+max(c)-min(c);%货架距离
            end
         end
        y(j,1)=y1;
    end
    [best_y,b]=min(y);
    best_x=x{b,1};
    %以最小化
    while(T>T_min)
        xx=[];
```

```
        yy=[];
        for j=1:sum
            y1=0;
            xx{j,1}=randperm(N);
            xx{j,1}=x{j,1}(1:length(S{i,1}));
            for k=1:size(XX,1)
                b=find(ismember(S{i,1},XX{k,2})==1);
                if b~=0
                    c=xx{j,1}(b);
                    y1=y1+max(c)-min(c);
                end
            end
            yy(j,1)=y1;
            %是否更新最优
            delta=yy(j,1)-y(j,1);
            if delta<0
                y(j,1)=yy(j,1);
                x(j,:)=xx(j,:);
            else
                p=exp(-delta/T);
                if R>rand
                    y(j,1)=yy(j,1);
                    x(j,:)=xx(j,:);
                end
            end
        end
        if min(y)<best_y
            [best_y,b]=min(y);
            best_x=x{b,1};
        end
        T=T*rate;
    end
    R{i,1}=best_x;
end
res=[];
for i=1:length(R)

res=[res;S{i,1},repmat(string(i),length(S{i,1}),1),string(R{i,1}')];
end

%导出数据
title={'ItemNo' 'GroupNo' 'ShelfNo'};
xlswrite('D:\dek\result2.csv', title,'sheet1');
xlswrite('D:\dek\result2.csv',res,'sheet1','A2')
```

附录三：

介绍：通过对货架号计算拣选路径和折线图的 Python 代码。

```
import csv
import matplotlib.pyplot as plt

def csv_read(path):
    with open(path, "r") as f:
        reader = csv.reader(f)
        res = []
        for row in reader:
            res.append(row)
        return res

def save_excel(res,y):
    # rows = list(zip(res.keys(), res.values()))
    # print(rows)
    rows=zip(y,res)
    row_1 = ['OrderNo', 'd']
    row_2 = ['所有批次的拣选距离总和为：',sum(res)]
    with open('拣选距离.csv', 'w', newline='')as csv_file:
        writer = csv.writer(csv_file)
        writer.writerow(row_1)
        writer.writerows(rows)
        writer.writerow(row_2)


def tor(dno, gno):   # 返回该订单的拣选距离
    good = []   # 存放每个订单的商品号
    shelf = []   # 存放每个人订单的货架号
    for i in ex3:
        if i[0] == dno:
            good.append(i[1])

    for i in good:
        for j in ex2:
            if i == j[0] and gno == j[1]:
                shelf.append(j[2])
    if (shelf):
        d = max(shelf) - min(shelf)
        res.append(d)
        y.append(dno)
```

```python
# 读取文件
ex1 = csv_read(r'result1.csv')    # 0 为 OrderNo 1 为 GroupNo
ex2 = csv_read(r'result2.csv')    # 0 为 ItemNo 1 为 GROUPNO 2 为 shelfNo
ex3 = csv_read(r'附件 1：订单信息.csv')
# 数据预处理
ex1.pop(0)
ex2.pop(0)
ex3.pop(0)
for i in ex1:
    i[1] = int(i[1])
for i in ex2:
    i[1] = int(i[1])
    i[2] = int(i[2])
for i in ex3:
    i[2] = int(i[2])
res = []    # 用于放置每个订单的拣选距离
y =[]
for d in ex1:
    tor(d[0], d[1])


print(res)
print(sum(res))
save_excel(res,y)

plt.plot(res[1:30],y[1:30],'b*--', alpha=0.5, linewidth=1)
plt.ylabel('订单号')
plt.xlabel('拣选距离')

plt.rcParams['font.sans-serif']=['SimHei']
plt.rcParams['axes.unicode_minus'] = False

plt.show()


plt.plot(res[131:160],y[131:160],'b*--', alpha=0.5, linewidth=1)
plt.ylabel('订单号')
plt.xlabel('拣选距离')
plt.show()

plt.plot(res[261:290],y[261:290],'b*--', alpha=0.5, linewidth=1)
plt.ylabel('订单号')
```

```
plt.xlabel('拣选距离')
plt.show()

plt.plot(res[791:820],y[791:820],'b*--', alpha=0.5, linewidth=1)
plt.ylabel('订单号')
plt.xlabel('拣选距离')
plt.show()
```

附录四：

介绍：运用长度优先的原则对拣选距离进行排序并指派工人的 Python 代码。

```
import csv


def csv_read(path):
    with open(path, "r") as f:
        reader = csv.reader(f)
        res = []
        for row in reader:
            res.append(row)
        return res


def save_excel(res):
    row_1 = ['OrderNo', 'GroupNo', 'WorkNo', 'TaskNo']
    rows = res
    with open('result3.csv', 'w', newline='')as csv_file:
        writer = csv.writer(csv_file)
        writer.writerow(row_1)
        writer.writerows(rows)


ex1 = csv_read(r'拣选距离 01.csv')
ex1.pop(0)
ex1.pop(len(ex1) - 1)
for i in ex1:
    i[1] = int(i[1])
ex1 = dict(ex1)

ex2 = csv_read('result1.csv')
ex2.pop(0)
for i in ex2:
```

```
        i[1] = int(i[1])
ex2 = dict(ex2)

f = 1
res = {}
ls = []
for i in ex2:
        if ex2[i] != f:
                res.update({f: ls})
                ls = []
                f = ex2[i]
        for j in ex1:
                if j == i:
                        ls.append((j, ex1[j]))
res.update({f: ls})
worker = 1
step = 1
res_l = []
print(res)

for i in range(1, len(res) + 1):
        res_p = res[i]
        res_p = sorted(res_p, key=lambda x: x[1], reverse=True)
        while res_p:
                if worker > 5:
                        worker = 1
                        step += 1
                res_l.append((res_p[0][0], i, worker, step))
                worker += 1
                res_p.pop(0)
        step = 1
        worker = 1

print(res_l)
save_excel(res_l)
```

附录五：

介绍：支撑文件文件目录。

question2.m
result1.csv
result2.csv
result3.csv
附件1：订单信息.csv
拣选距离.csv
拣选距离.py
问题一.py
问题三.py
A题论文.docx